Federal University of ABC (UFABC)
Course: CCM313-EEI203: Performance Analysis of Networks and Systems, 2026.3

Project Work Plan

Rafael Paniccia Cassetta, 21202520618
Submission Date: October 25, 2026

1. Project Title

PicoClaw vs. OpenClaw: Performance Analysis of Go and TypeScript AI Agents on Apple Silicon M4

2. Objective

The primary objective of this project is to quantitatively measure, compare, characterize, and understand the runtime efficiency, latency trade-offs, resource consumption, and orchestration overhead of two contrasting personal AI agent architectures: the ultra-lightweight edge framework PicoClaw (Sipeed native Go) and the modular desktop/server framework OpenClaw (TypeScript on Node.js). The project establishes a strictly controlled, isomorphic benchmark on Apple Silicon (M4 SoC, 24 GB Unified Memory) with equalized neural inference layers to:

- **Evaluate:** Measure end-to-end task completion latency (T_task), Time to First Token (TTFT), Inter-Token Latency (ITL), Resident Set Size (RSS), cold-start launch duration (T_cold), SoC package energy consumption (E_pkg), and tool-calling execution throughput (R_tool).
- **Compare:** Quantify the operational performance differentials between dynamic JavaScript/V8 execution (OpenClaw) and native statically compiled goroutine execution (PicoClaw).
- **Characterize:** Decouple and isolate the overhead introduced strictly by the agent orchestration runtime (JSON validation, prompt templating, IPC socket dispatch, and garbage collection) from the underlying neural model inference time.
- **Understand:** Determine how Apple Silicon's Unified Memory Architecture (UMA) mediates memory bus contention and energy efficiency when local neural inference co-executes with agent orchestration layers.

3. Motivation

Deploying autonomous personal AI agents locally on client hardware guarantees user privacy, deterministic local file access, offline autonomy, and elimination of recurring cloud API fees. However, current agent frameworks adopt contrasting engineering approaches:

- **Desktop/Server-Centric Runtimes (OpenClaw in Node.js/TypeScript):** Emphasize broad modularity, dynamic typing, and expansive plugin ecosystems (NPM), but suffer from large memory footprints (>165 MB idle, ~450 MB under multi-turn loops), large default system prompts (~16K tokens across 54 tools), V8 JIT bootstrap overhead, and generational Garbage Collection (GC) pauses.
- **Edge/Embedded Runtimes (PicoClaw in Go):** Emphasize bare-metal efficiency, single-binary static compilation, predictable non-generational concurrent tri-color mark-sweep GC, sub-10 ms cold starts, and minimal resident memory (~14 MB idle, <40 MB peak).

4. Research Question

How do PicoClaw (Go) and OpenClaw (Node.js/TypeScript) compare in latency, memory footprint, energy consumption, and tool-calling throughput under equalized LLM workloads on Apple M4?

5. Metrics

The study defines direct hardware/software measurements and derived analytical indicators across n = 30 independent repetitions per treatment:

Table 1: Summary of Performance Metrics, Units, and Collection Methods

Metric Name	Symbol	Unit	Type	Collection / Calculation Method
Task Completion Latency	T_task	ms	Direct	POSIX clock_gettime_ns monotonic clock
Time to First Token	TTFT	ms	Direct	Timestamp delta on first SSE streaming chunk
Inter-Token Latency	ITL	ms/tok	Direct	Mean interval between consecutive SSE chunks
Cold-Start Launch Latency	T_cold	ms	Direct	hyperfine v2.0.0 (30 runs, execve to exit/ready)
Resident Set Size & Phys RAM	RSS	MB	Direct	Darwin libproc RUSAGE_INFO_V4 & hyperfine
Hardware CPU Counters	PMC	count	Direct	RUSAGE_INFO_V4 ri_instructions & ri_cycles
Package Energy Consumption	E_pkg	Joules	Direct/Cal.	RUSAGE_INFO_V4 ri_billed_energy + M4 TDP model
Orchestration Overhead Ratio	Φ_orch	%	Derived	((T_task - T_neural) / T_task) × 100%
Tool Dispatch Throughput	R_tool	ops/sec	Derived	N_tools / T_dispatch_total
Energy-Delay Product	EDP	J·s	Derived	E_pkg × (T_task / 1000)
Memory Amplification Factor	α_mem	ratio	Derived	RSS_peak / RSS_idle

6. Workload

Table 2: Quantitative Workload Tier Specifications (W1–W4)

Tier	Level	Operational Focus	Prompt Tokens	Output Tokens	Tool Calls
Tier 1	W1 (Low)	Single-Turn Cognitive Reflection (No tools)	128	64	0
Tier 2	W2 (Med)	3-Step Sequential Tool Chain (list_dir, grep_logs, calc_stats)	256 (~850 cum.)	128	3
Tier 3	W3 (High)	Vector Memory RAG (10,000 docs, 100k-param 384d head)	1,536	256	1 (RAG)
Tier 4	W4 (Stress)	10-Turn Autonomous ReAct Diagnostic Loop	512 -> 2,048	10 × 64 (640)	10

7. Tools

Table 3: Experimental Toolchain, Validated Versions, and Purpose

Tool Name	Validated Version	Category	Purpose in Evaluation
PicoClaw	v0.3.1 / Go 1.23.2	System Under Test	Edge native compiled Go agent orchestrator
OpenClaw	v2026.10.7 / Node 26.12.1	System Under Test	Desktop/Server TypeScript agent orchestrator
llama.cpp	b11887 (Apple Metal)	Neural Inference	Local OpenAI-compatible LLM server on 127.0.0.1:8080
Llama-3.2-3B-Instruct	GGUF Q4_K_M (1.92 GiB)	Neural Model	Equalized 3B LLM weights across both agents
hyperfine	v2.0.0 (arm64)	Measurement	High-precision cold-start & PMC benchmarking
Darwin libproc / psutil	RUSAGE_INFO_V4 / v7.2.2	Measurement	Kernel RSS, physical footprint, instructions, cycles, energy
Python Harness (uv)	CPython 3.12.3 / uv 0.13.5	Workload Driver	AsyncIO driver, datasets, SciPy stats, Matplotlib plots

8. Parameters, Factors, and Levels

- **Factor A (Agent Architecture - 2 Levels):** A1: OpenClaw (Node.js/TypeScript V8 runtime); A2: PicoClaw (Go native static binary).
- **Factor B (Workload Complexity - 4 Levels):** B1: W1 (Reflection); B2: W2 (3-Step Tool Chain); B3: W3 (10k Vector RAG); B4: W4 (10-Turn ReAct Loop).
- **Factor C (Execution Warmth - 2 Levels):** C1: Cold Start (process spawned per request); C2: Warm Steady-State (persistent process).

9. Experimental Environment

All experiments execute on a dedicated Apple Mac mini (Apple M4 SoC: 10 CPU cores [4P+6E], 10-core Metal GPU, 16-core ANE, 24 GB Unified LPDDR5X Memory @ 120 GB/s, macOS 26.6.2 Build 25G83) with sleep disabled via /usr/bin/caffeinate -dims.

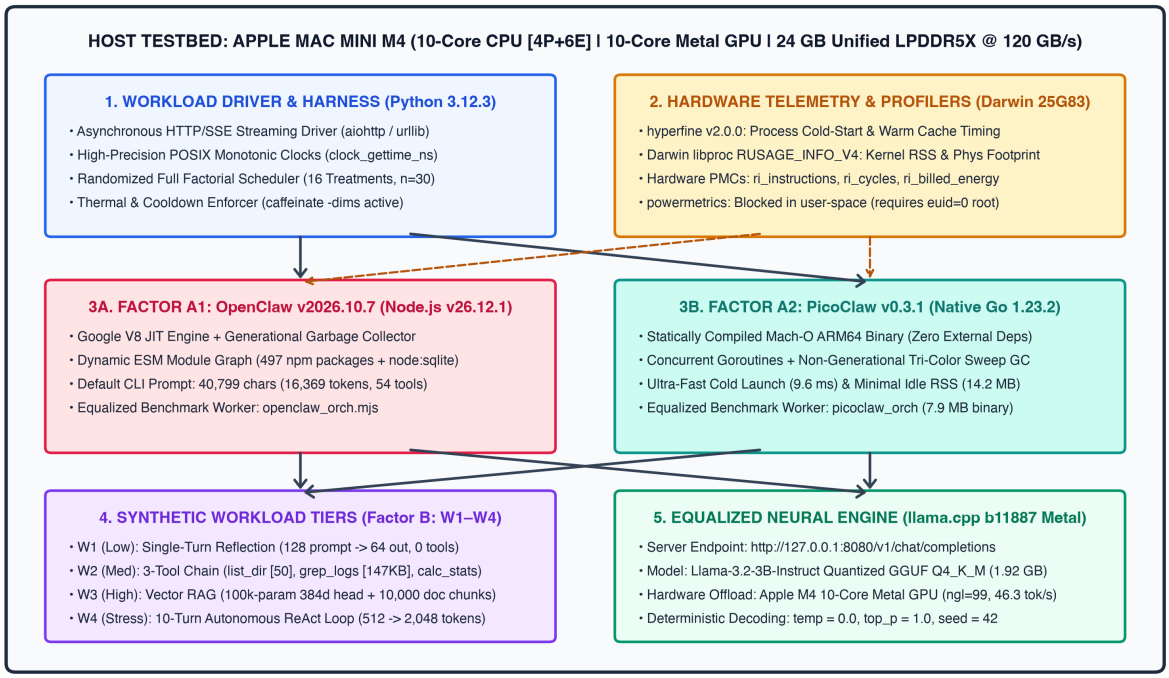


Figure 1: System Topology on Apple Mac mini M4 (24 GB Unified Memory).

10. Experiments

Table 4: Experimental Execution Plan and Statistical Design (480 Total Observations)

Experiment	Primary Metrics	Treatments Evaluated	Repetitions	Statistical Evaluation
Exp 1: Startup & Footprint	T_cold, RSS_idle, RSS_peak, PMCs	T1, T2, T9, T10	30 per treatment (120)	Welch's t-test, 95% CI
Exp 2: Latency & Overhead	T_task, TTFT, ITL, Φ_orch	T1 through T16	30 per treatment (480)	Two-way ANOVA, Pairwise t-test
Exp 3: Energy & Power	E_pkg, P_avg, EDP	T1 through T16	30 per treatment (480)	95% Confidence Intervals, EDP
Exp 4: Tool Throughput	T_dispatch, R_tool, α_mem	T3, T4, T7, T8, T11, T12, T15, T16	30 per treatment (240)	Throughput scaling & ratio

11. Use of Artificial Intelligence

Google Antigravity AI assistant (powered by Gemini) assisted in refining the research question and title for conciseness, structuring the factorial design according to Raj Jain's methodology, and formatting the document. All experimental design decisions and hardware validations were supervised by Rafael Paniccia Cassetta (RA: 21202520618).

12. References

1. Jain, R. (1991). The Art of Computer Systems Performance Analysis. John Wiley & Sons.
2. Liu, X., et al. (2024). AgentBench: Evaluating LLMs as Agents. ICLR 2024.
3. Sipeed. (2026). PicoClaw: Ultra-lightweight Go-based Agent Framework (v0.3.1). GitHub Repository.
4. Steinberger, P., & OpenClaw Foundation. (2026). OpenClaw Autonomous Personal Assistant (v2026.10.7). GitHub Repository.